AI ACCELARETOR - פרויקט גמר

טעינת הנתונים + EDA

טווח הזמן שאסקר בפרוייקט הוא בין התאריכים 23.09.2018 - 23.09.2022

נתוני המנייה (stock_data):

- המניה שבחרתי היא מניית שופיפיי (Shopify). שופיפיי היא פלטפורמת האיקומרס (מסחר דיגיטלי) המובילה בעולם, והיא עוזרת לבעלי עסקים לפתוח חנויות אינטרנטיות בקלות.
- מחיר המנייה משקף את הביטחון של בעלי המניות והמשקיעים בשופיפיי, ובשל הובלתה בתחום, בתחום האיקומרס כולו.

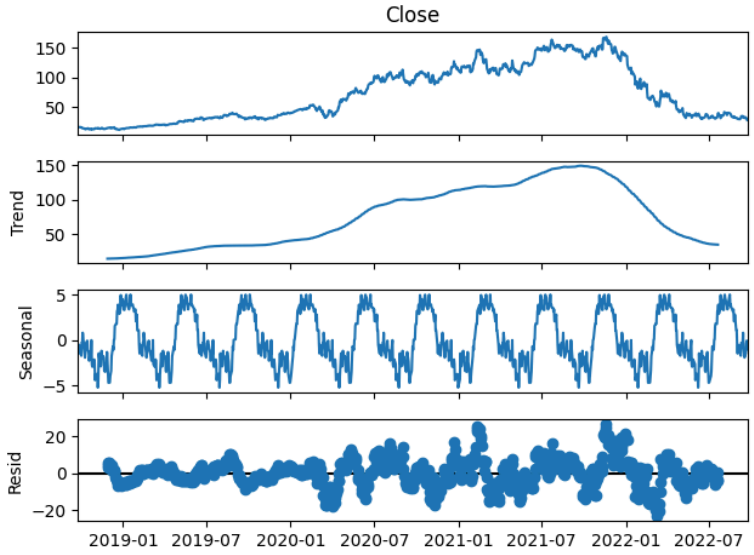
בשלב ה-EDA, בחרתי להתייחס לכמה נתונים מעניינים במנייה ולהציג אותם:

- השתמשתי בפקודת describe על מנת להסיק נתונים סטטיסטיים על הנתונים וגיליתי:
  - המחיר הממוצע של המניה (לנתוני הסגירה) הוא $71.609~, אולם אפשר לראות גם שסטיית התקן עומדת על $46.38~, מה שמצביע על שונותה של המנייה וכמה המחיר לא היה יציב לאורך תקופת הזמן הזאת.
  - את הגיוון במחירים של המנייה ניתן לראות גם עבור טווח המחירים, המחיר המינימלי בתקופת הזמן הזאת היה $11.91~, ואילו המחיר המקסימלי עומד על שיר של $169.06~.
  - בנוסף, מנתוני המסחר של המנייה (Volume), ניתן לראות כי ממוצע המניות שנסחר מדי יום הוא 21.6 מיליון מניות, אולם גם כאן סטיית התקן עומדת על 14.6 מניות מדי יום.

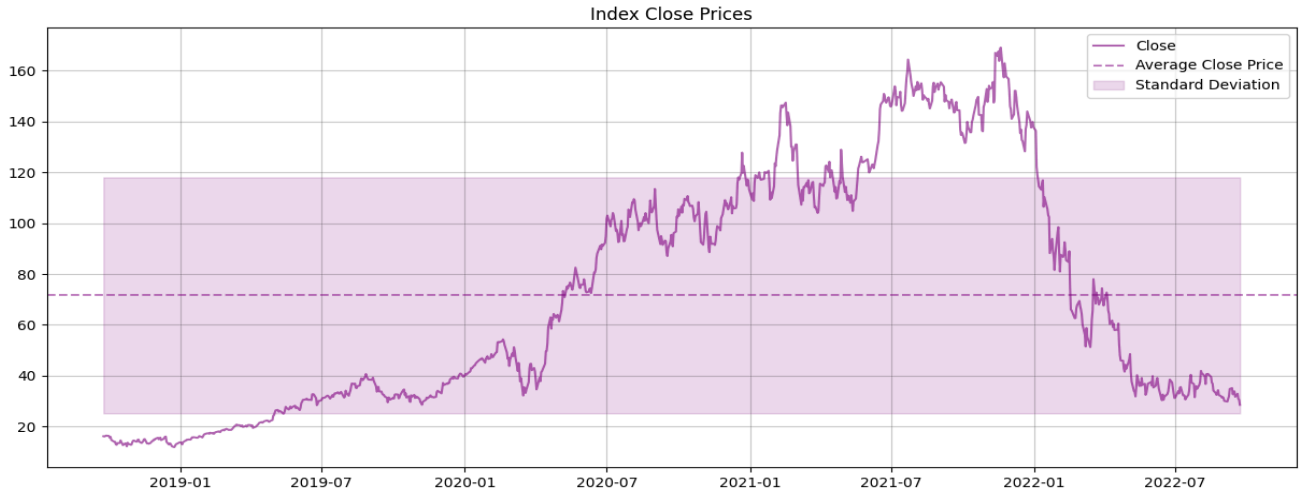
```
# Generate descriptive statistics for the stock data (mean, std, min, max, etc.)
stock_data.describe()
```

	Close	High	Low	Open	Volume
count	1007.000000	1007.000000	1007.000000	1007.000000	1.007000e+03
mean	71.609979	73.303438	69.824745	71.660892	2.174099e+07
std	46.384810	47.301162	45.475659	46.499072	1.402607e+07
min	11.910000	12.287000	11.764000	11.895000	4.713000e+06
25%	31.906500	32.759501	31.146600	31.839000	1.232400e+07
50%	54.437000	58.054001	53.407001	55.256001	1.785800e+07
75%	110.759499	114.450001	108.200001	111.134499	2.778400e+07
max	169.059998	176.291794	168.509506	171.800003	1.136080e+08

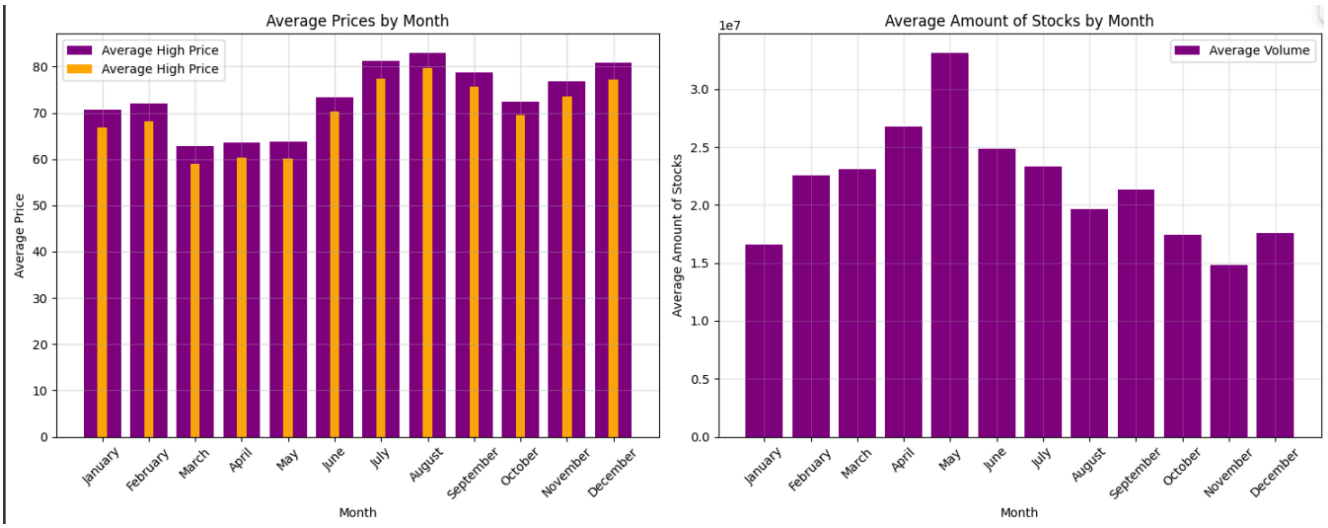
- לאחר מכן, הצגתי את מחירי הסגירה של המניה לאורך התקופה בגרף כדי לקבל המחשה של המחירים, והראתי שיש טרנד של עלייה ואז ירידה לאורך הזמן באמצעות seasonal_decompose (שנראה כמו נחש שאוכל פיל 🙂):
  - לאורך התקופה נראה כי הייתה עלייה חדה במנייה לאורך הזמן, ישנם הרבה "שפיצים" שמעידים על התנודתיות של המניה ועל הסיכון בלהשקיע בה.



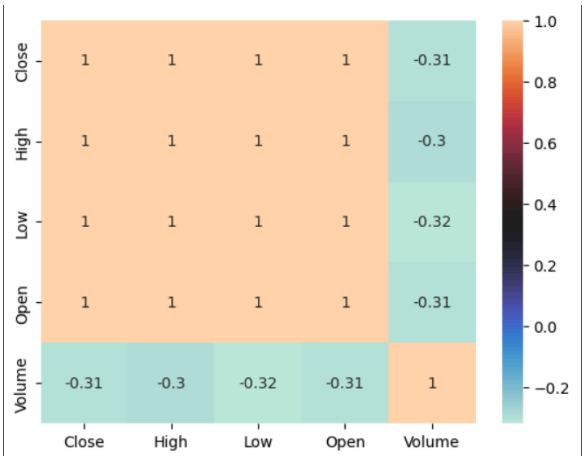
- בתקופה של פברואר 2020 ועד לספטמבר 2021 (בערך), ישנה עלייה חדה במחירי המנייה, שכנראה נובע ממגיפת הקורונה. (אולי בעצם זה שאנשים נשארו בבית הם קנו יותר בדיגיטל או שלמדו לפתוח בעצמם חנויות כדי להתפרנס).



- לאחר מכן, הצגתי את המחירים הממוצעים של מחירי המקסימום והמינימום מידי חודש, לצד הנפח הממוצע לחודש:
  - ניתן לראות שהמחירים הנמוכים ביותר היו בחודשים מרץ-מאי, כנראה בשל כך שהקורונה רק התחילה באותה תקופה ולא הייתה לה השפעה חזקה באותם החודשים, וההשפעה הגדולה החלה רק לתוך השנה, דבר שמחזק את זה הוא העובדה שחודשים יולי-ספטמבר הם החודשים עם המחירים הגבוהים ביותר.
  - ניתן לראות על פי הגרף השני מאי הוא החודש עם נפח המסחר הממוצע הגבוה ביותר, אולי כי מירב הפעילות במניה היה לאחר עליית הקורונה וגם לאחר דעיכתה (סיום הסגרים שהתרחש באותה תקופה).



- הראתי את הקורלציות של העמודות בדאטאפריים באמצעות seaborn, ונוכחתי לראות שלרוב העמודות יש קורלציה, בשל הטווח הקטן שיש בין מחירי המנייה מדי יום, ואולם לעמודת הנפח אין קורלציה מספיק חזקה עם שאר הנתונים, לכן החלטתי לוותר עליה בשלבי המודלים.

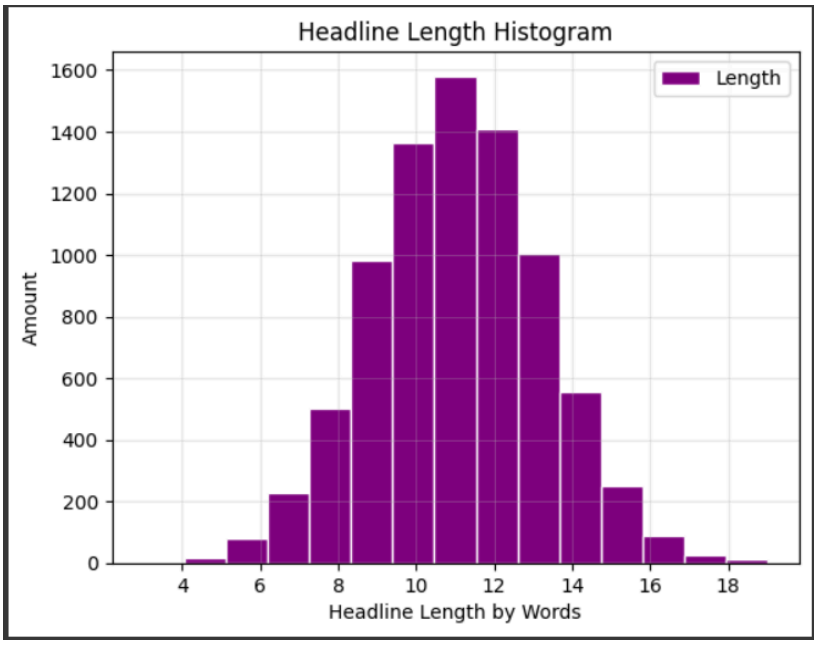


נתוני החדשות (news_data):

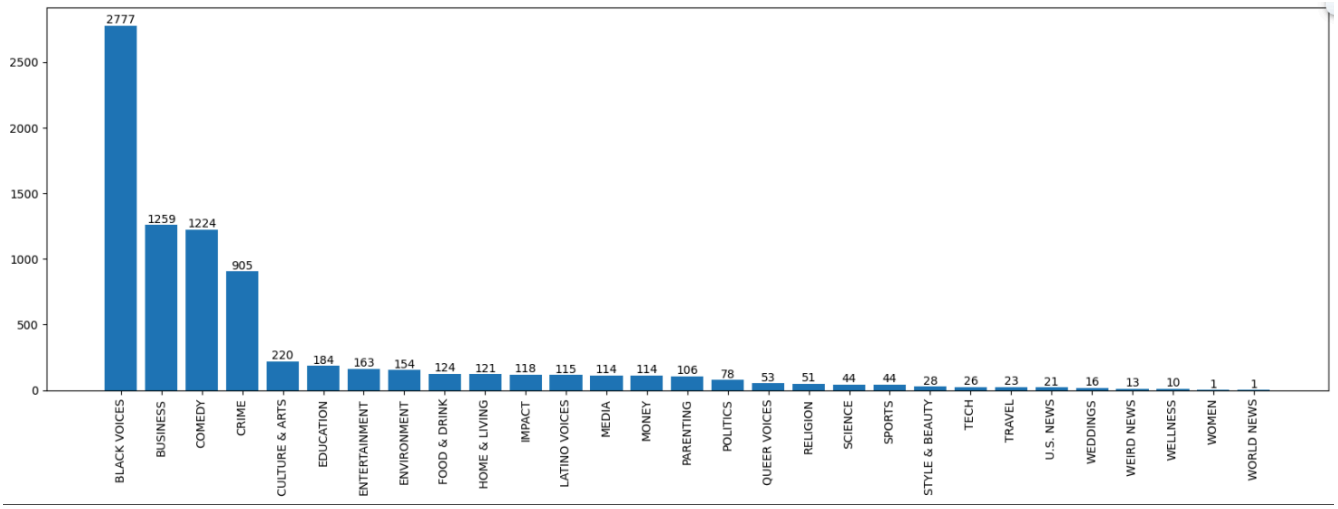
- מאגר החדשות שבחרתי הוא News Category Dataset מתוך Kaggle, שאוגר בתוכו כ-210,000 כתבות מאתר החדשות HuffPost, מתוכן 10,000 (בקירוב) כתבות רלוונטיות לתקופת הזמן שאתייחס אליה בפרויקט.
- הנתונים הספציפיים שאשתמש בהם עבור כל כתבה הם כותרת ('headline'), קטגוריה ('category') ותאריך ('date').

עבור נתוני החדשות, אלה הגרפים שבחרתי להציג:

- השתמשתי בהיסטוגרמה על מנת להציג את אורכי הכותרת (במילים), וזהיתי שיש התנהגות נורמלית בין אורכי הכותרות (הגיוני בשל משפט הגבול המרכזי):

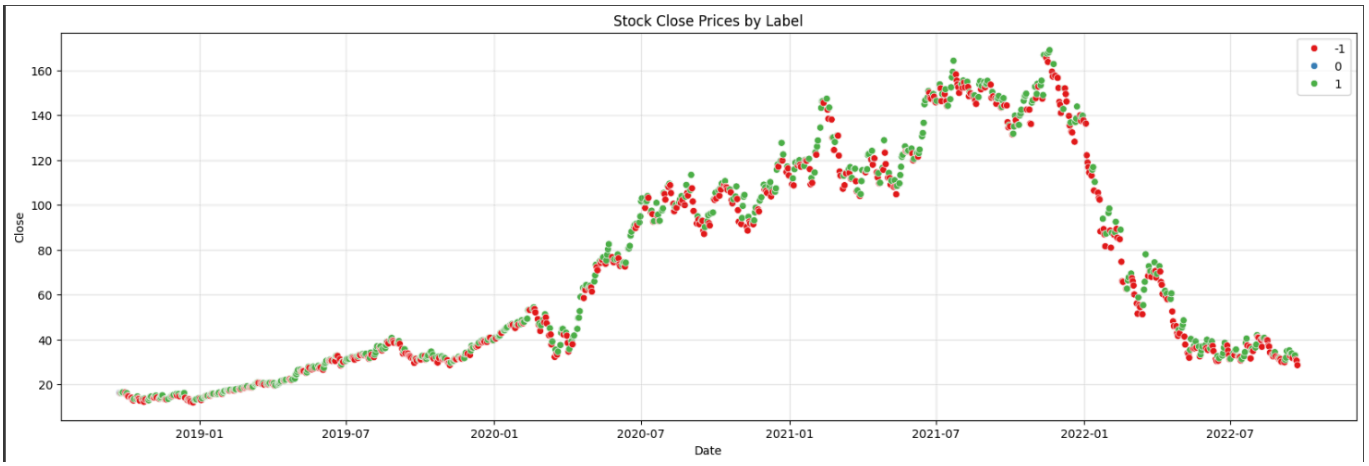


- בנוסף, בחרתי להציג את כמות הכותרות מכל קטגוריה וגיליתי:
  - 5 הקטגוריות המובילות הן: Black Voices, Business, Comedy, Crime, Culture & Arts.
  - אבחר לשלב את הקטגוריות בכותרות של הכתבות בשלב ה-NLP על מנת לראות האם זה ישפר את ביצועי המודל שלי.

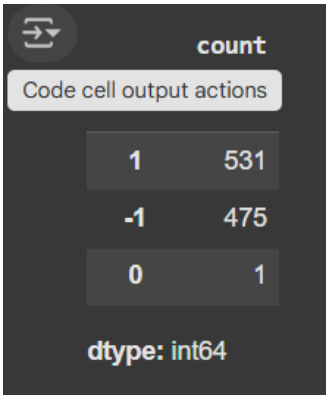


עמודת Label (שינוי):

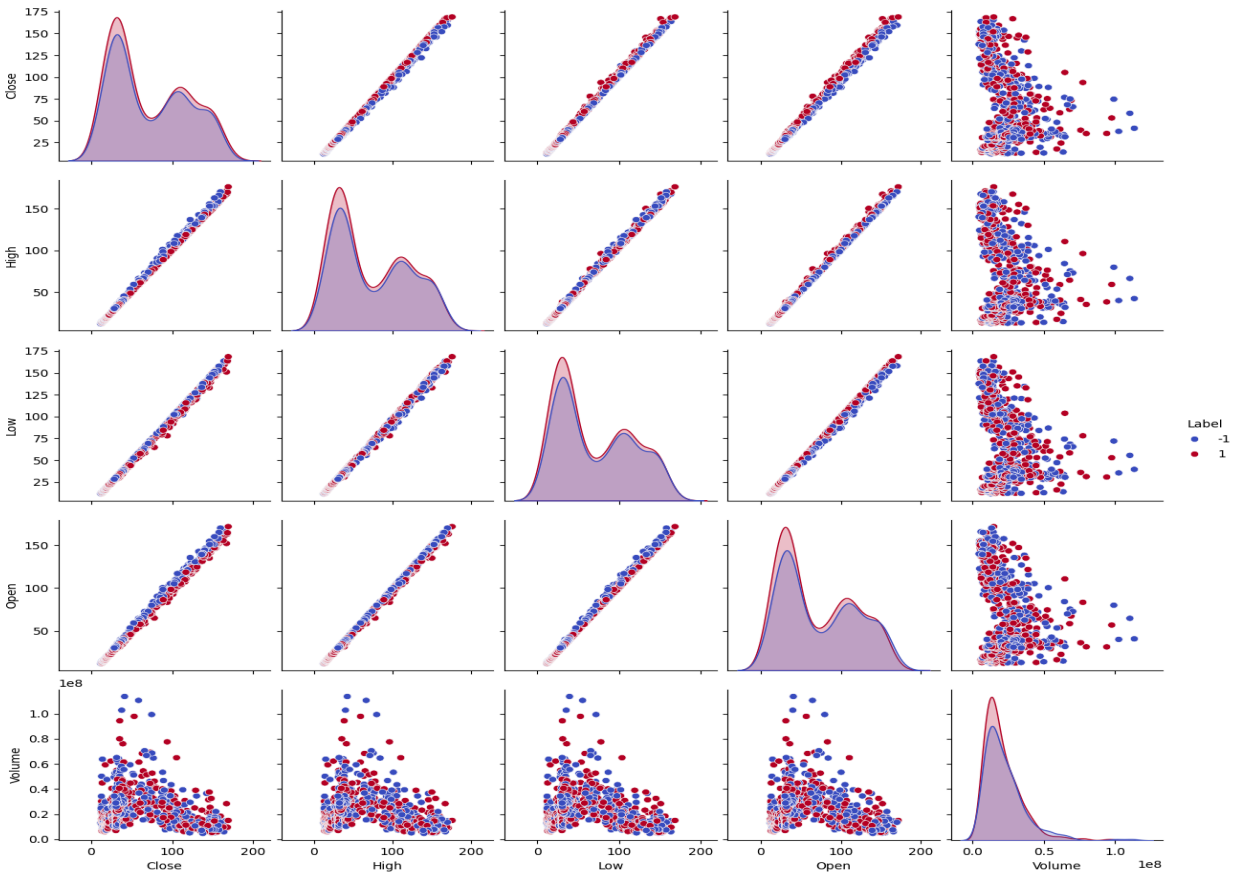
- לצורך חישוב השינוי בין ימים עוקבים, השתמשתי בפונקציה pct_change() שמחשבת עבור ערכים בדאטאפריים את אחוז השינוי מהיום הקודם.
- לאחר מכן, בעזרת seaborn, הצגתי בגרף שוב את מחירי הסגירה של המנייה, אך הפעם סיווגתי את המחירים לפי עמודת השינוי (עלייה - 1, ללא שינוי - 0, ירידה - -1).
- לאחר שראיתי את התוצאות האלה, בחרתי לא להוסיף עמודות נוספות שמבוססות על התאריך של המחיר למודלים שלי, שכן לא ראיתי תבניות או clusters מספיק חזקים.



- ניתן לראות גם שהצבע הכחול, שמסמל שאין שינוי, כמעט לא נראה על הגרף, לאחר שווידאתי שיש רק ערך אחד כזה, הסרתי את השורה המתאימה לו מהדאטאפריים כדי לשפר את התוצאות. (בתמונה: מספר הרשומות מכל סוג שינוי בדאטאפריים).



- בנוסף, השתמשתי בpairplot כדי לראות האם יש קשרים בין הנתונים של stock_data לעמודת השינוי, וגיליתי כמה דברים:
  - נראה שבין העמודה Volume לבין שאר העמודות אין קשר חזק, ובנוסף הוא לא מצביע על הפרדה שיש בין הלייבלים השונים.
  - אולם נראה שלמשל בין עמודת Open לעמודת Close יש הפרדה ליניארית כמעט בין סוגי הלייבלים.



- על סמך המידע הזה, המודלים שבחרתי הם Logistic Regression ו-SVM.

מודלים קלאסיים

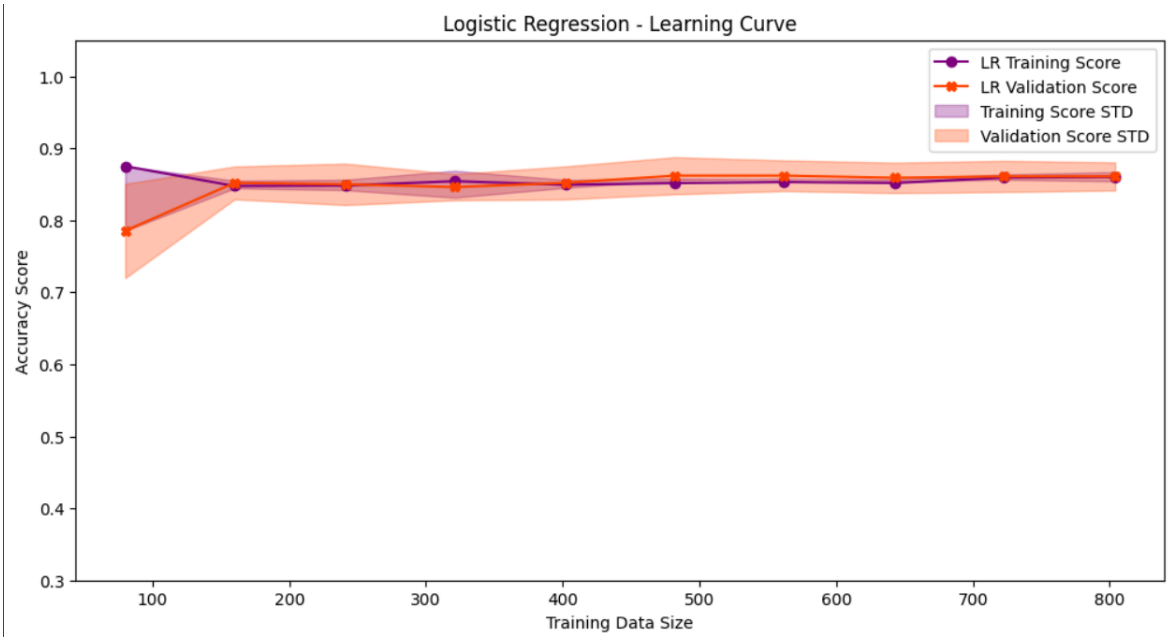
מודל רגרסיה לוגיסטית:

- המודל הראשון שבחרתי הוא Logistic Regression, בהתבסס על ההפרדה הכמעט ליניארית שראיתי בין הנתונים בשלב הקודם. את פרמטר penalty הפכתי ל-l1, מכיוון שהוא גורם לפרמטרים לא חשובים להתאפס, ומכיוון שראיתי קשר חזק בין Open ו-Close לבין הלייבלים, החלטתי להשתמש בו. את C קבעתי להיות מספר שהוא גבוה יחסית, כדי שהמודל יקלוט ויילמד דפוסים שהם יותר מורכבים כדי שלא תהיה התאמה שהיא ליניארית ממש (גם במחיר של Overfitting, למרות שזה לא המקרה כאן).

```
# Creating the Logistic Regression model.
LR_model = LogisticRegression(solver='saga', penalty = 'l1' , max_iter = 5000, C = 100,  random_state = 42)

# Train the Logistic Regression model on the training data.
LR_model.fit(X_train, y_train)

# Make predictions on the training and testing data.
lr_y_val, lr_y_pred = LR_model.predict(X_train), LR_model.predict(X_test)

# Print the accuracy scores for the training and testing sets, along with the delta.
print(f'LR Regression accuracy for train: {round(accuracy_score(lr_y_val, y_train), 3) }, for test: {round(a
```

```
LR Regression accuracy for train: 0.857, for test: 0.861, delta: -0.442%
```

- בתוצאות המודל קיבלתי אחוז דיוק של ~85.7% עבור נתוני האימון, ואילו ~86.1% עבור נתוני המבחן, ההפרש הוא כ-~0.4% לטובת נתוני המבחן דווקא, מה שגורם לחשוב שהמודל התאמן טוב יותר על נתוני המבחן, ושלא היו מספיק נתונים עבור נתוני האימון.
- בשלב הבא בחרתי להציג את עקומת הלמידה של המודל, כדי להראות איך לאורך גדלי נתונים שונים המודל מצליח ללמוד ולהשתפר, וגיליתי:
    - אחוז הדיוק הממוצע של המודל על נתוני האימון נשאר יציב לאורך הזמן, וכך גם סטיית התקן. בנוסף עקומת הלמידה של המודל על נתוני המבחן מראים שסטיית התקן כן קטנה לאורך הזמן, מה שמעיד על מודל יציב, אך עם זאת הסטייה נשארת פחות או יותר קבועה (במחברת הדפסתי גם את סטיית התקן המקסימלית, הממוצעת והאחרונה).



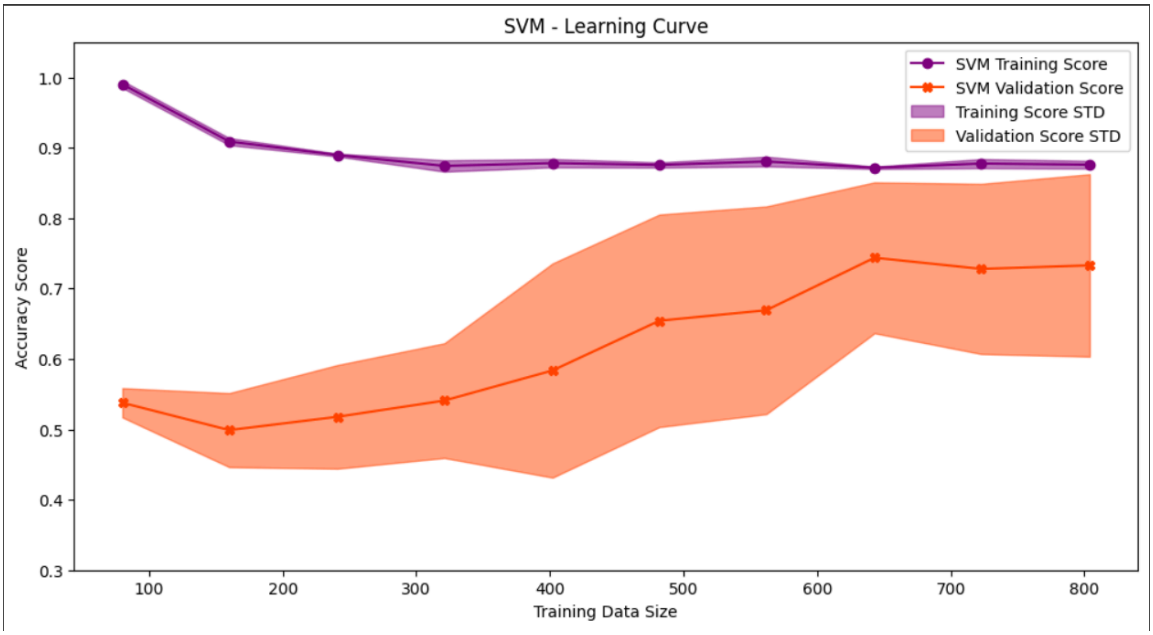
מודל SVM:

- המודל השני שבחרתי הוא מודל SVM, שכן הוא מסתמך על מציאת הישר/המישור עם השוליים הכי רחבים כדי להפריד בין קבוצות/לייבלים שונים באמצעות Support Vectors. השתמשתי בRBF כדי שההפרדה לא תהיה רק ליניארית, ועל מנת לאפשר גמישות למודל. את הערך C, שאחראי לענישה גם כאן, קבעתי למספר גבוה יחסית, כדי לאפשר למודל שלי לדייק יותר ולא לאפשר חריגות לתוך המרחב שיצר ה-SVM.

```
# Initialize the Support Vector Machine (SVM) model.
SVM_model = SVC(kernel='rbf', C = 1e5, random_state = 42,)

# Train the SVM model on the training data.
SVM_model.fit(X_train, y_train)

# Make predictions on the training and testing data.
svm_y_val, svm_y_pred = SVM_model.predict(X_train), SVM_model.predict(X_test)

# Print the accuracy scores for the training and testing sets, along with the del
print(f'SVM accuracy for train: {round(accuracy_score(svm_y_val, y_train), 3) },
```

```
SVM accuracy for train: 0.871, for test: 0.876, delta: -0.559%
```

- בתוצאות המודל קיבלתי אחוז דיוק של ~87.1% עבור נתוני האימון, ואילו ~87.6% עבור נתוני המבחן, ההפרש הוא כ-~0.6% לטובת נתוני המבחן דווקא. אפשר לחשוב שהמודל הזה טוב יותר, אבל עקומת הלמידה מלמדת אחרת.
- לפי עקומת הלמידה, ניתן לראות שאחוז הדיוק של המודל על נתוני האימון מתחילים באחוז גבוה ולאט יורדים, שזאת ההתנהגות שאנו אמורים לצפות לה בעקומה למידה, אולם סטיית התקן של נתוני האימון היא קטנה לכל אורך המודל, מה שיכול לרמז על כך שהמודל לומד בצורה מדויקת מדי את נתוני האימון. מהצד השני, אחוז הדיוק של המודל על נתוני המבחן מתחיל ב-~53% ועולה לאורך הדרך, אולם גם סטיית התקן עולה איתו, מה שיכול לרמז על מודל שאינו יציב באמת. גם אחוז הדיוק הממוצע של בסוף תהליך הלמידה עומד על ~72%, משמעותית נמוך מאחוזי הדיוק של המודל על נתוני המבחן (~90%) ומאחוזי הדיוק של מודל הרגרסיה הלוגיסטית.



השוואת המודלים

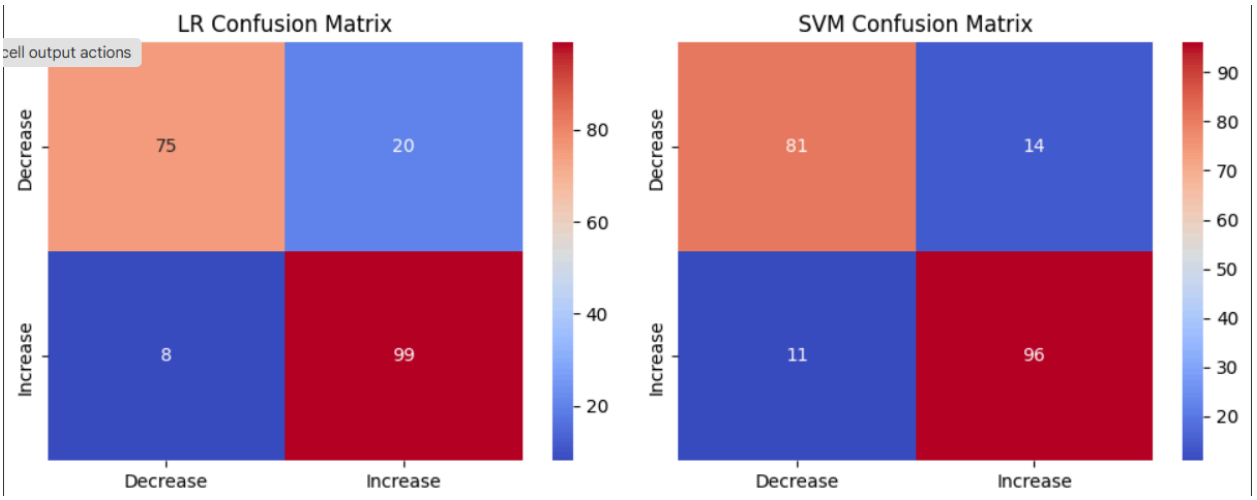
- הדרך הראשונה בה בחרתי להשוות בין שני המודלים היא ציון סטיית התקן של המודלים בתהליך עקומת הלמידה:
  - ניתן לראות שסטיית התקן הממוצעת של מודל ה-SVM על נתוני האימון הוא ~10%, ואילו של הרגרסיה הלוגיסטית עומד רק על ~2.6%, ולכן מודל הרגרסיה הליניארית הוא יותר יציב ולכן עדיף, שכן השיפור באחוזי הדיוק של מודל ה-SVM מזעריים.

```
LOGISTOC
Maximum STD in the process: 6.528
Average STD in the process: 2.665
Last STD in the process: 1.94

_________________
SVM
Maximum STD in the process: 15.209
Average STD in the process: 10.362
Last STD in the process: 12.947
```

- המדדים האחרים שהשתמשתי בהם הם classification report ומפת חום של confusion matrix.

- לפי הנתונים ניתן לראות שהמודל SVM השיג ציונים טובים יותר כמעט בכל פרמטר אפשרי, כולל f1-score, מה שמצביע על כך שהמודל (ספציפית לנתונים האלה), יותר מדויק ומאוזן. בנוסף לפי מפת החום, החסרונות בביצועי מודל ה-SVM לעומת הרגרסיה הלוגיסטית הוא בזיהוי העליות כעליות (96 לעומת 99 במודל ה-LR) ובנוסף במיסקלאסיפיקציה של נתונים בתור עלייה כשבעצם הם סווגו כירידה.

```
Logistic Regression Classification Report
              precision    recall  f1-score   support

           0       0.90      0.79      0.84        95
           1       0.83      0.93      0.88       107

    accuracy                           0.86       202
   macro avg       0.87      0.86      0.86       202
weighted avg       0.87      0.86      0.86       202

SVM Classification Report
              precision    recall  f1-score   support

           0       0.88      0.85      0.87        95
           1       0.87      0.90      0.88       107

    accuracy                           0.88       202
   macro avg       0.88      0.87      0.88       202
weighted avg       0.88      0.88      0.88       202
```



מודל LSTM (חיזוי מבוסס זמן)

- עבור החלק של חיזוי הזמן, השתמשתי בפונקציה של Tensorflow ליצירת חלונות הזמן, על מנת להכניס אלמנט של מורכבות למודל באמצעות shuffle ויצירת batchים לשיפור הלמידה. בחרתי בחלון זמן של 60 ימים ו-batch של 6 ערכים.
- בחרתי להשתמש בכל הנתונים לחיזוי (מלבד Volume כי ראינו שהוא בעייתי), ביצעתי Scaling לנתונים באמצעות MinMaxScaler.
- עבור בניית המודל, בחרתי להשתמש בשתי שכבות LSTM, שכן LSTM היא רשת נוירונים שמתאימה ללמידה עבור נתונים שמבוססים סדרתיות, היא מאפשרת זיכרון של הנתונים לאורך זמן, וכך משפרת את ביצועי המודל. לא השתמשתי בשכבות נוספות מלבד שכבת ה-Output, שאמורה לחזות מחיר ולכן יש בה נוירון אחד.

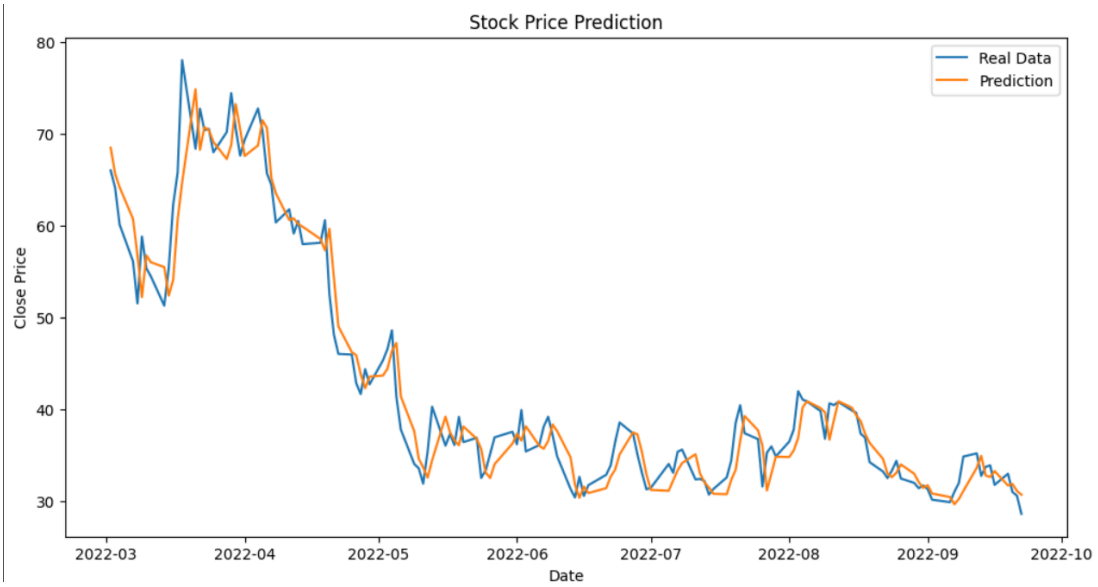
```python
# Define the time series model using Keras Sequential API
ts_model = Sequential([
    Input(shape = (WINDOW_SIZE, X_fit.shape[1])),
    LSTM(128, return_sequences= True),
    LSTM(128,),
    Dense(1, activation=None)
])


# Compile the model
ts_model.compile(loss="mse",
                 optimizer = tf.keras.optimizers.Adam(learning_rate = 0.001),
                 metrics = ['mae'])

# Define callbacks for early stopping and learning rate reduction
callbacks = [
    tf.keras.callbacks.EarlyStopping(monitor='val_mae', patience = 7, restore_best_weights = True,),
    tf.keras.callbacks.ReduceLROnPlateau(monitor = 'val_mae', patience = 4, factor = 0.3333),
    ]

# Train the model
history = ts_model.fit(
    train_dataset,
    validation_data = valid_dataset,
    epochs = 40,
    batch_size = 64,
    callbacks = callbacks,
    verbose = 2,
)
```

- על מנת להבין את ביצועי המודל בצורה טובה יותר, השתמשתי במספר מדדים:
  - ראשית, הדפסתי את ציון ה-MSE וה-MAE הטובים ביותר של המודל על ידי שימוש בפעולה evaluate על סט הולידציה (המבחן), השתמש ב-MAE כדי לקבל הבנה טובה יותר של הטעות.

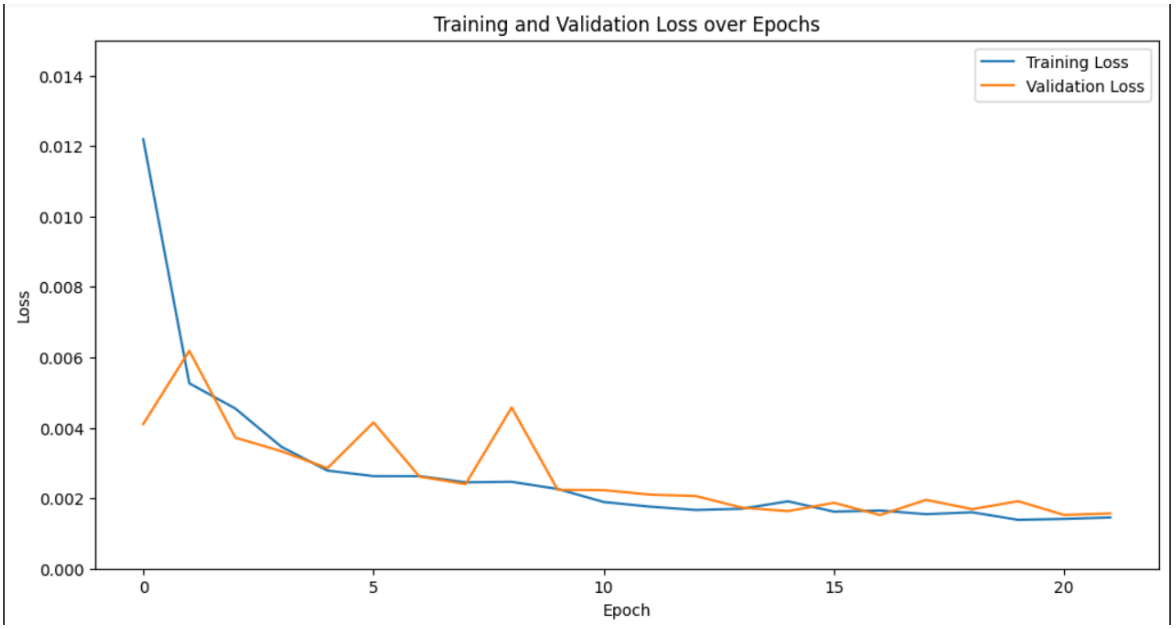
```python
ts_model_scores = ts_model.evaluate(valid_dataset, return_dict = True)
# Print the best MSE and the epoch it was achieved
for key, value in ts_model_scores.items():
    print(key, ':', round(value, 4))
```

```
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 24ms/step - loss: 0.0014 - mae: 0.0296
loss : 0.0014
mae : 0.0285
```

  - לאחר מכן, הצגתי על תקופת הזמן שבחנתי (20% מסך הנתונים) את חיזוי המודל על נתוני המבחן:



- כמו כן, הצגתי את שיעור הירידה של פונקציית ההפסד של המודל על נתוני האימון (בכחול) ועל נתוני המבחן (בכתום).



- לבסוף חישבתי (בעזרת TensorFlow כמובן) את R2 כדי להראות כמה המודל שלי חזה מדויק בהתבסס על הניחוש הממוצע.

```
[109] print("R2 Score for the model:", r2_score(stock_data['Close'][split_
```

```
R2 Score for the model: 0.9480232885873305
```

מודל NLP:

טוקניזציה + הכנת הנתונים:

- בתור שלב ראשון, ניקיתי את הטקסטים באמצעות שימוש בטוקנייזר וב-Lemmetizer. מעבר לכך בחרתי להחליף את כל הנקודות (.) במחרוזת ריקה, כך שמילים כמו U.S לא ייחשבו כשתי מילים שונות ב-Word Index שלי. את הניקוי של הטקסטים ביצעתי על שרשור של הכותרת והקטגוריה של כל כתבה.

שלב הבנייה:

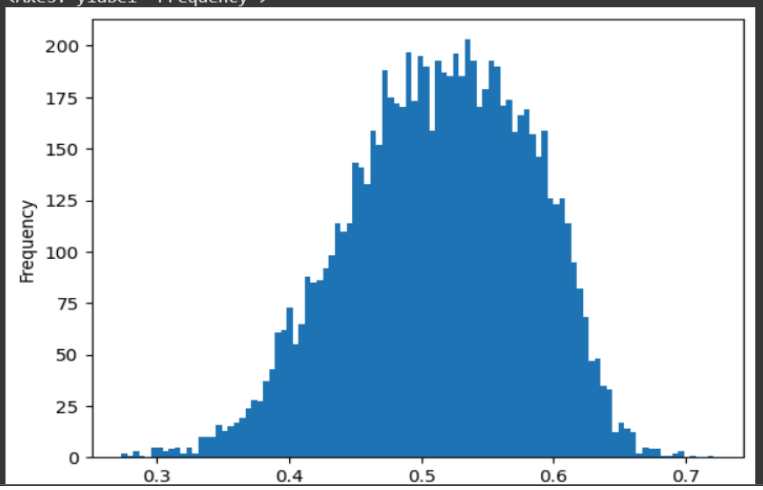
- עבור המודל עצמו, השתמשתי בשכבת Embedding על מנת להפוך את הרצפים לווקטורים מספריים. השתמשתי בשכבת GRU אחת כי המידע הוא סדרתי, ועל כן דורש התחשבות בנתונים לאורך הסדרה, בשל הנתונים שהם קצרים יחסית (אורכי הסדרות מילים) ומטעמי חישוב העדפתי אותו על LSTM. ובנוסף הוספתי שכבת Dense כדי להוסיף מורכבות למודל. בסוף המודל, השתמשתי בשכבת למבדה, משום שתוצאות המודל נתנו לי טווחי מספרים קטנים בסדר גודל של אלפית (0.001), על כן הגדרתי פונקציה משלי שמכפילה את הערך החזוי ב-100 ומבצעת עליו sigmoid, כדי לאפשר טווח תוצאות רחב יותר.

```python
# Import the random module for reproducibility
import random
# Set the random seed for consistent results
random.seed(42)

# Define the NLP model using Keras Sequential.
nlp_model = Sequential([
    # Embedding layer to represent words as vectors
    Embedding(input_dim = len(tokenizer.word_index) + 1, output_dim = 8), # input_dim - תיקון שלנו
    GRU(256),
    # Dense layer with LeakyReLU activation
    Dense(64,),
    Dense(1, ),
    tf.keras.layers.Lambda(lambda x: tf.keras.activations.sigmoid(x * 100 ))
]
)

# Compile the model
nlp_model.compile(
    optimizer = tf.keras.optimizers.Adam(learning_rate = 1e-03),
    loss = "mse",
    metrics = ['mae']
)

# Make predictions on the news data sequences to create sentiments, and setting in the dataframe.
sentiment = nlp_model.predict(tf.convert_to_tensor(new_news_data['sequence'].to_list()))
new_news_data['sentiment'] = sentiment
```

- עבור המודל עצמו, השתמשתי בשכבת Embedding על מנת להפוך את הרצפים לווקטורים מספריים. השתמשתי בשכבת GRU אחת כי המידע הוא סדרתי, ועל כן דורש התחשבות בנתונים לאורך הסדרה, בשל הנתונים שהם קצרים יחסית (אורכי הסדרות מילים) ומטעמי חישוב העדפתי אותו על LSTM. ובנוסף הוספתי שכבת Dense כדי להוסיף מורכבות למודל. בסוף המודל, השתמשתי בשכבת למבדה, משום שתוצאות המודל נתנו לי טווחי מספרים קטנים בסדר גודל של אלפית (0.001), על כן הגדרתי פונקציה משלי שמכפילה את הערך החזוי ב-100 ומבצעת עליו sigmoid, כדי לאפשר טווח תוצאות רחב יותר.
- על מנת לוודא את הטווח, הדפסתי רשימה של ערכים שיעזרו לי לקבל תחושה של ביצועי המודלת ביניהם הסנטימנט המינימלי, המקסימלי, הממוצע וסטיית התקן. זאת כדי לוודא שהטווח הוא בין 0 ל-1, אך לא קיצוני לאחד מהצדדים.

```python
print(new_news_data['sentiment'].min())
print(new_news_data['sentiment'].max())
print(new_news_data['sentiment'].std())
print(new_news_data['sentiment'].mean())
new_news_data['sentiment'].sort_values().plot(kind = 'hist', bins = 100)
```

```
0.2737261652946472
0.7205066680908203
0.06821222603321075
0.51412266
<Axes: ylabel='Frequency'>
```



הוספת הסנטימנט למודלים

- על מנת להוסיף את הסנטימנט לדאטאפריים של המניות, חישבתי (לא לבד, עם Pandas) את הציון הממוצע לכל יום.

```
[222] new_stock_data_w_sent = new_stock_data.copy()

      news_by_date = new_news_data.groupby('date')
      new_stock_data_w_sent['Sentiment'] = news_by_date[['sentiment']].mean()
```

מודלים קלאסים

- לאחר שהוספתי את עמודת הסנטימנט וביצעתי שוב הערכה של המודלים הקלאסיים באמצעות אימון וחיזוי, קיבלתי את הנתונים הבאים:
  - נראה כי ציון הדיוק עבור נתוני המודל של הרגרסיה הלוגיסטית עלה מעט, מציון של ~86.1% לציון של 86.6%, עלייה של ~0.58%.
  - אולם עבור מודל ה-SVM ההצלחה לא מסחררת, דיוק המודל נפל מעט וירד מ-~87.6% ל-~85.1%, מה שמכריע את מודל ה-LR כמנצח הגדול מבין המודלים.

```
LOGOSTIC
accuracy for train: 0.858
accuracy for test: 0.866
delta: -0.8

_________
SVM
accuracy for train: 0.894
accuracy for test: 0.851
delta: 4.3

_________
```

מודל	רגרסיה לוגיסטית	SVM
נתוני האימון	בלי סנטימנט - 85.6 עם סנטימנט - 85.8	בלי סנטימנט - 87.1 עם סנטימנט - 89.4
ביצועים בלי סנטימנט - נתוני המבחן	בלי סנטימנט - 86.1 עם סנטימנט - 86.6	בלי סנטימנט - 87.6 עם סנטימנט - 85.1
שינוי באחוזים (נתוני המבחן)	+0.58%	-2.85%

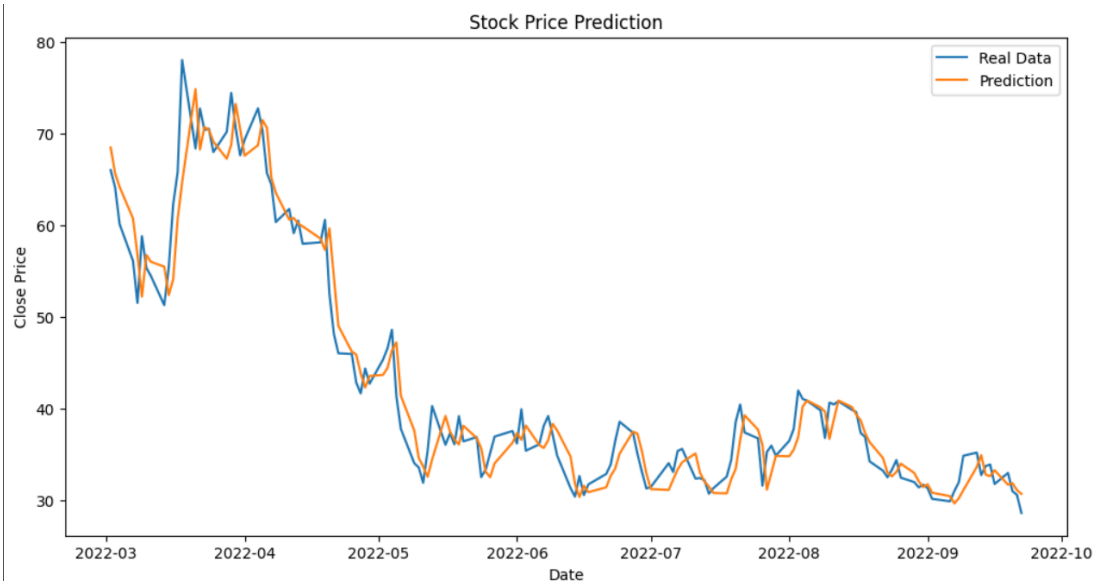
מודל ה-LSTM

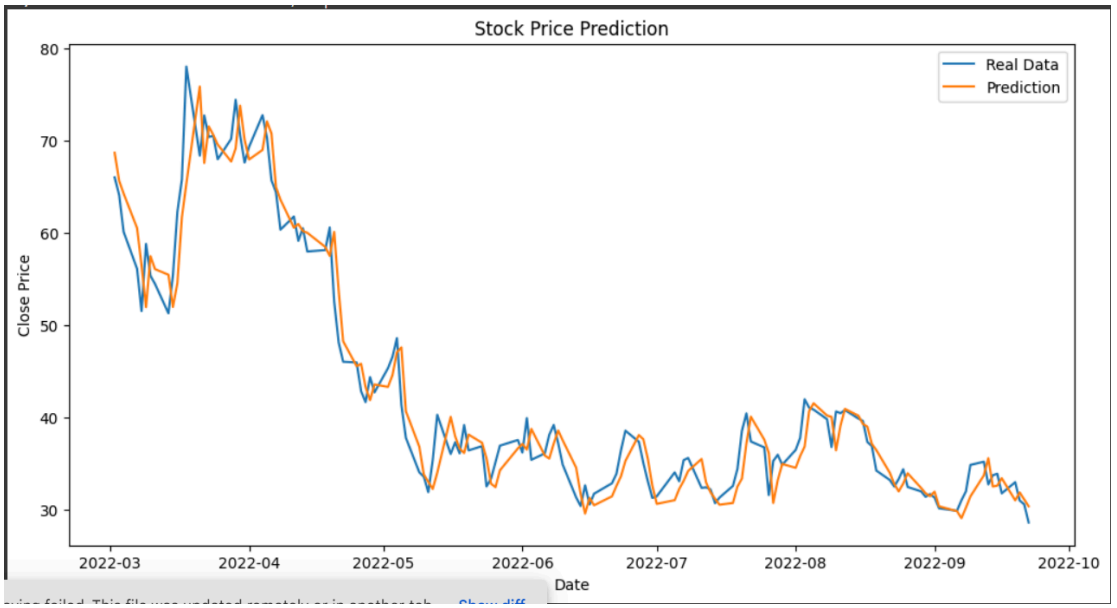
- עבור מודל ה-LSTM, בניתי מחדש את אותו המודל כדי לאמן אותו מחדש ולאפס את המשקלים (כדי למנוע למידה של המודל על בסיס המידע הקיים), וקיבלתי את התוצאות הבאות:
  - עבור ציון ה-MSE וה-MAE, קיבלתי לאחר השימוש ב-Sentiment תוצאות פחות טובות, X לעומת ציון MAE של Y עבור שימוש במודל ללא הסנטימנט.

```
ts_model_scores = ts_model.evaluate(valid_dataset, return_dict = True)
# Print the best MSE and the epoch it was achieved
for key, value in ts_model_scores.items():
    print(key, ':', round(value, 4))
```

```
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 24ms/step - loss: 0.0014 - mae: 0.0296
loss : 0.0014
mae : 0.0285
```

- עבור החיזוי, אפשר לראות כי הוא כמעט זהה לחיזוי הקודם. מטעמי נוחות צירפתי בתמונה הראשונה את המודל הקודם, ובתמונה השנייה את המודל עם הסנטימנט.





- הגורם הסופי הוא ציון ה-R2 שפחות טוב במקצת אחרי השימוש בסנטימנט.

```
ts_model_scores = ts_model_new.evaluate(valid_dataset_new, return_dict = True)
for key, value in ts_model_scores.items():
    print(key, ':', round(value, 4))
```

```
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0023 - mae: 0.0378
loss : 0.0014
mae : 0.0288
```

לסיכום, ניתן לראות שביצועי המודלים לאחר השימוש בסנטימנט פחתו, אם כי לא בצורה משמעותית, אפשרות אחת היא שסט הנתונים של החדשות לא מספיק בכל קורלציה לנתוני המניה של שופיפיי, ולכן לא הייתה השפעה רבה.